Kolumbus and Agder Timetable History

1 Project Statement and Outcome

Bus routes, line numbers and timetables in Rogaland and Agder change during the year, but Entur only publishes the current plan, and every new export replaces the old one. We build a pipeline that saves Entur's NeTEx exports for **Kolumbus (KOL)** and **Agder Kollektivtrafikk (AKT)**, cleans them with PySpark and turns them into a history of the planned service. **Bounded outcome:** for 2026 exports we deliver (1) planned departures per line, operator, day and month, and (2) a list of lines that were added, removed or renumbered, where a change is only called a renumbering if the operator confirms it. Real-time data, passenger numbers and other operators are out of scope.

2 Dataset

- **Source:** Entur's national NeTEx timetable export, one ZIP file per day (about 280 MB). It is open data under NLOD and needs no API key.
- **Suitability:** the XML is deeply nested (lines, routes, stop patterns, trips, passing times, calendars and national stops) and needs many joins. The nine historical snapshots have 263,757 trips and 8.3 million passing times, and everything must be re-run when a rule changes, so Spark is a good fit.
- **Limitations:** the data shows plans, not what actually ran. Each export only covers a rolling period, some exports are broken (1–2 February 2026), AKT repeats some IDs in different files, times after midnight use day offsets, and on-demand trips have time windows instead of departures.
- **Refresh pattern:** Entur publishes a new export every day. We collect yesterday's export on days 1, 4, 7 and so on of each month. Our evaluation sample is nine historical snapshots from the 15th of each month (January–September 2026), plus the first scheduled snapshot (8 November 2026) as a tenth observation. Extra exports from the start of each month cover the first days of each month.

3 Bronze Design

- **Ingestion:** small Python command-line tools (standard library only) for historical, scheduled, manual ZIP and calendar-support ingestion. All of them use the same selection and checks.
- **Preservation:** we keep only the KOL and AKT XML and the national stops file, with the original bytes, inside one checked ZIP per run. Every run gets a new folder (`run-RUN_ID-ATTEMPT`) and nothing is ever overwritten.
- **Traceability:** each run writes a manifest (source link, size, MD5, SHA-256, trigger type), a lineage file with the checksum of every XML file, and a validation report. Checksums, ZIP CRCs, XML parsing and every reference are checked; a failure stops the run and keeps the files as evidence.
- **Scheduling and deployment:** GitHub Actions starts the job on days 1, 4, 7 and so on of each month (`23 8 */3 * *`, so not exactly every 72 hours) and on every push. It runs on a self-hosted runner on the school server, and the data is stored in `/srv/dat535/data`, outside the code folder.

4 Silver Design

- **Grain:** one row per source occurrence (operator, file, frame and position), identified by a SHA-256 occurrence key, because the same ID can appear in several files. Passing times are one row per trip and stop number, and `journey_service_dates` has one row per trip per operating day.
- **Model:** Spark first writes 16 staging Parquet tables, one per NeTEx type. Silver then builds `lines`, `routes`, `journey_patterns`, `journeys`, `passing_times`, `calendar_dates` and `journey_service_dates`, plus `quarantine_*` tables. Each stop is found through the chain passing time → pattern point → scheduled stop → stop assignment → quay → national stop place.
- **Cleaning:** we never delete rows. A lookup only accepts exactly one match; no match is "unresolved" and more than one is "ambiguous". Every row gets a list of reason codes and an `is_valid` flag, and bad rows are also copied to quarantine tables. Identical shared stop definitions are merged, and conflicting ones are quarantined.
- **Standardisation and enrichment:** times are converted to seconds from the start of the operating day, including the day offset (00:10 + 1 day = 87,000 s), and a `lag` window checks that they never go backwards. Calendars (weekdays, periods, single dates and exclusions) are expanded into real dates, and on-demand trips are marked separately.
- **Quality evidence:** a count check makes sure Silver has exactly the same number of lines, trips and passing times as staging. A profile per operator counts quarantined, unresolved and missing values, and an independent script re-reads the XML without Spark for four representative lines (normal, overnight, calendar exception, on-demand) and compares the results. A quality gate stops Gold if more than 1% of rows are rejected.
- **Spark implementation:** Spark 4.2's XML reader with fixed schemas and `FAILFAST`; DataFrame API (`join`, `groupBy`, `posexplode`), window functions (`count`, `row_number`, `lag`) and Spark SQL expressions such as `where("is_valid AND NOT is_flexible")`. Every table is written, read back and counted.
- **Performance:** we only tune based on evidence from real runs: the small calendar tables are handled on the driver, `spark.sql.shuffle.partitions` is set to 4 for the 2-core server, cached tables are stored on disk and released after use,

broadcast joins are turned off where they ran out of memory, and each snapshot runs in a fresh Spark process with 2 GiB of memory.

- **Scalability:** the large tables only use distributed DataFrame operations, so the code can run on a cluster by changing `--master`. Checkpoints store a checksum of the code and the input, so only the changed steps run again, and later exports are handled by using the newest export that covers each date.

5 Gold Design

Consumer: a transport analyst or journalist who wants to know how much service is planned and what changed in the network. Gold is built only from valid, fixed trips, and only if the quality gate passes. `daily_line_service` and `daily_network_service` give departures per line and per operator per day. `is_source_covered` separates a day with no buses from a day with no data. `line_window_summary`, `line_stop_coverage` and `excluded_service_dates` complete the picture. The Gold total must match Silver exactly. Route history adds SCD Type 2 line versions and operator-confirmed renumberings. All of this is shown in a read-only Streamlit GUI.

6 End-to-End Delivery

- **Reproducibility:** all code and rules are in Git with fixed package versions. Every step writes a JSON manifest, and a SQLite checkpoint registry records each attempt with checksums of its code and input. There are 175 unit and Spark tests.
- **Environments:** during development we use a local `data/` folder. On the school server we use `/srv/dat535/data` (set by `DAT535_DATA_ROOT`), with separate Python environments for Spark and the GUI, and Java 17 for Spark.
- **Automation:** the GitHub Actions workflow runs tests → ingestion → staging, Silver and Gold → route history → restart of the GUI. If a step fails, the steps after it do not run.
- **Monitoring:** the Actions log and a 30-day report artifact, the manifests, the checkpoint registry and the GUI's Data checks page show the status, quarantined rows and quality results.
- **Ownership:** one of us leads historical data, Silver, Gold and route history, and the other leads ingestion, automation, deployment and the GUI. We share Silver quality checks, design decisions, testing and the report.